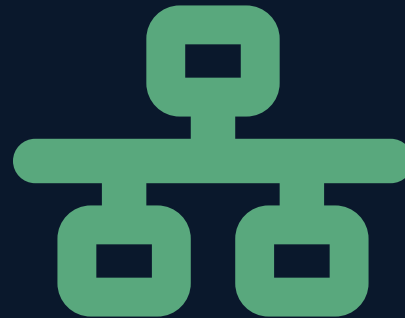


PORTADA

Arquitectura de Software

Semana 8 — Catálogo POSA: SOA, Arquitectura Hexagonal

Universidad de Antioquía · Ingeniería de Sistemas · 2554608 · 2026-2



APERTURA

**Un memo interno de dos páginas.
El origen de AWS.**



El "API Mandate" de Amazon, 2002

Empresa	Amazon
Año	2002
Quién lo ordenó	Jeff Bezos, fundador y entonces CEO de Amazon
Forma	Memo interno dirigido a todos los equipos de ingeniería (nunca publicado oficialmente por Amazon)
Cómo se conoce hoy	Por el relato de Steve Yegge , ex-ingeniero de Amazon y Google, en su "Platform Rant" de 2011
Efecto posterior	Catalizador directo del nacimiento de AWS (Amazon Web Services) en 2006

El texto exacto del memo nunca se hizo público; lo que conocemos es el relato —ampliamente citado en la industria— de alguien que trabajó dentro de Amazon cuando ocurrió.

CASO · CONTEXTO

Un monolito que ya no podía crecer así

A comienzos de los 2000, Amazon había pasado de ser una tienda de libros a un comercio electrónico con decenas de equipos trabajando sobre **el mismo sistema gigante y fuertemente interconectado**.

- Equipos distintos accedían **directamente a las bases de datos de otros equipos**, sin pasar por ninguna interfaz formal.
- Un cambio en el esquema de una tabla podía romper, sin aviso, código de un equipo completamente distinto.
- No existía una forma estándar de saber qué funcionalidad ofrecía cada parte del sistema ni cómo consumirla de forma segura.

El acoplamiento entre partes del sistema —cada una dependiendo de los detalles internos de las demás— se había vuelto el principal freno para escalar la ingeniería de la empresa.

CASO · LA ORDEN

Reglas sin excepción, para todos los equipos

Según el relato de Yegge, el memo de Bezos establecía —en esencia— un conjunto de reglas obligatorias para **todos** los equipos de Amazon:

- Toda funcionalidad debe exponerse **únicamente a través de interfaces de servicio** bien definidas.
- **Prohibido** el acceso directo a bases de datos de otros equipos, la comunicación por memoria compartida o cualquier otro "atajo" interno.
- Los equipos solo pueden comunicarse entre sí **a través de la red**, llamando a esas interfaces de servicio.
- Cada interfaz debe diseñarse pensando en que, algún día, podría exponerse a desarrolladores externos —sin importar si eso estaba planeado o no.

El memo terminaba, según el mismo relato, con una advertencia directa: quien no cumpliera sería despedido. No hay una transcripción oficial de esa frase exacta, pero es la parte más citada de la historia.

CASO · CONSECUENCIA

Los mismos servicios internos, abiertos al mundo

La disciplina forzada por el mandato tuvo un efecto que probablemente nadie planeó desde el inicio: para 2006, Amazon ya tenía **decenas de servicios internos con interfaces bien definidas, probadas y estables** —cómputo, almacenamiento, bases de datos.

Ese mismo año, Amazon lanzó **Amazon Web Services (AWS)**, ofreciendo a desarrolladores externos acceso a esos mismos servicios (S3 para almacenamiento, EC2 para cómputo) que ya existían puertas adentro.

AWS no se construyó desde cero como producto: nació de exponer, a clientes externos, la misma disciplina de servicios que el mandato de 2002 había obligado a construir puertas adentro.

TRANSICIÓN

De una orden interna a un patrón con nombre propio

Lo que el mandato de Bezos impuso por decreto —servicios de negocio con interfaces bien definidas, sin atajos ni acoplamiento directo entre equipos— es exactamente lo que el catálogo POSA describe como un estilo arquitectónico formal.

Ese estilo se llama Arquitectura Orientada a Servicios, o SOA.

CONCEPTO

SOA: el sistema como servicios de negocio

SOA (Service-Oriented Architecture) organiza el sistema como un conjunto de **servicios de negocio** —unidades de funcionalidad con sentido propio para el negocio (facturación, inventario, envíos, pagos)— que se pueden **componer** entre sí para construir procesos más grandes.

- Cada **servicio** es una pieza de software autocontenida que resuelve una capacidad de negocio concreta y la expone a través de una interfaz.
- Los servicios se comunican entre sí **sin conocer los detalles internos** de cómo están implementados los demás.
- Un proceso de negocio grande (ej. "procesar un pedido") se arma **orquestando** varios servicios más pequeños.

Los cuatro principios de un buen servicio SOA

Principio	Qué significa
Reutilización	El mismo servicio (ej. autenticación) lo pueden consumir muchos procesos de negocio distintos, sin duplicar su lógica
Bajo acoplamiento	Un servicio puede cambiar su implementación interna sin obligar a cambiar a quienes lo consumen
Contrato explícito	La forma de usar el servicio —qué operaciones ofrece, qué datos espera y devuelve— queda documentada en una interfaz formal (el "contrato"), no se infiere del código interno
Autonomía	Cada servicio controla y despliega su propia lógica de forma independiente de los demás

Contrato: la definición formal —independiente de la tecnología usada por dentro— de qué operaciones ofrece un servicio y con qué datos, exactamente lo que el mandato de Amazon obligó a formalizar en cada equipo.

CONCEPTO · VENTAJAS Y LIMITACIONES

Lo que SOA gana — y lo que cuesta a escala

Ventajas	Limitaciones
Reutilización real entre equipos: un servicio bien diseñado evita reconstruir la misma capacidad varias veces	La gobernanza de contratos —quién puede cambiar una interfaz y cómo se avisa a quienes la consumen— se vuelve un problema serio cuando hay decenas de servicios
Los equipos pueden evolucionar su servicio internamente sin coordinar cada cambio con el resto de la empresa	El versionado de contratos (qué pasa cuando un servicio necesita cambiar su interfaz sin romper a sus consumidores) exige disciplina y procesos formales
Permite componer procesos de negocio nuevos combinando servicios ya existentes	La comunicación por red entre servicios agrega latencia frente a llamadas directas dentro de un mismo proceso

TRANSICIÓN

Cambiamos de escala: del sistema completo, a un solo servicio

SOA resuelve cómo se organizan y comunican servicios **a nivel de sistema o de empresa completa**. Pero hay otra pregunta, igual de importante, que SOA no responde: **¿cómo se organiza el código dentro de uno solo de esos servicios** para que no termine, con el tiempo, atado a su base de datos o a su framework?

Esa pregunta la responde otro patrón del catálogo POSA: la Arquitectura Hexagonal.

CASO · FICHA TÉCNICA

Alistair Cockburn y "Ports and Adapters", 2005

Autor	Alistair Cockburn, consultor e ingeniero de software, coautor del Manifiesto Ágil (2001)
Año	2005
Nombre original	"Ports and Adapters" (Puertos y Adaptadores)
Nombre popular	Rebautizado después como " Hexagonal Architecture ", por el diagrama de hexágono con el que Cockburn lo ilustraba
Publicación	Blog personal de Cockburn, no en un libro formal del catálogo POSA

CASO · MOTIVACIÓN

La lógica de negocio, atrapada entre capas

Cockburn describió su frustración con las arquitecturas en capas tradicionales: en la práctica, la **lógica de negocio** terminaba mezclada con código de interfaz de usuario o con detalles de acceso a la base de datos.

- Para probar automáticamente una regla de negocio, era necesario levantar la base de datos o simular la interfaz gráfica completa.
- Cambiar de motor de base de datos, o de framework de interfaz, obligaba a tocar código que en teoría era "solo negocio".
- El diseño en capas por sí solo **no impedía** que el negocio dependiera de la infraestructura, aunque en teoría debía evitarlo.

Cockburn buscaba una forma de diseñar el núcleo del sistema para que fuera imposible que dependiera de sus detalles técnicos externos —y, de paso, fácil de probar sin mocks complicados.

CASO · INFLUENCIA POSTERIOR

La idea que siguió apareciendo después

La propuesta de Cockburn no se quedó aislada: influyó directamente en enfoques posteriores muy conocidos en la industria.

- **Clean Architecture**, de Robert C. Martin ("Uncle Bob"), publicada en 2012, retoma la misma idea central —el negocio en el centro, protegido de la infraestructura— con círculos concéntricos en vez de un hexágono.
- La forma de estructurar aplicaciones con **DDD (Domain-Driven Design, "Diseño Guiado por el Dominio")** —donde el modelo de negocio ocupa el centro del diseño y todo lo demás se organiza alrededor de él— comparte la misma inspiración.

No se desarrollan aquí como casos completos: basta con reconocer que "proteger el negocio de la infraestructura" es una idea que atraviesa varias propuestas influyentes de arquitectura, no solo la de Cockburn.

CONCEPTO

Puertos y adaptadores

La **Arquitectura Hexagonal** aísla la lógica de negocio (el **núcleo** o **dominio**) de los detalles de infraestructura —bases de datos, interfaces de usuario, APIs externas— usando dos conceptos:

- **Puerto:** una interfaz que el propio núcleo define, describiendo qué necesita del mundo exterior (ej. "guardar un pedido") o qué ofrece hacia afuera, sin decir cómo se hace.
- **Adaptador:** una implementación concreta que conecta un puerto con una tecnología real (ej. un adaptador que implementa "guardar un pedido" usando PostgreSQL, otro que lo hace con MongoDB).



CONCEPTO

El núcleo no depende de nada externo

La regla central del patrón es una **inversión de dependencias**: el núcleo define los puertos que necesita, y son los adaptadores —afuera— quienes dependen del núcleo, nunca al revés.

- **Dentro del hexágono:** el dominio y su lógica de negocio, sin ninguna referencia a bases de datos, frameworks web ni librerías externas.
- **En el borde del hexágono:** los puertos —las interfaces que el dominio expone o necesita.
- **Fuera del hexágono:** los adaptadores —UI web, API REST, base de datos, cola de mensajes— cada uno implementando un puerto.

Se puede reemplazar cualquier adaptador —cambiar de base de datos, de framework de UI— sin tocar una sola línea del núcleo.



CONCEPTO · VENTAJAS Y LIMITACIONES

Lo que Hexagonal gana — y lo que cuesta

Ventajas	Limitaciones
El dominio se prueba de forma aislada , sin levantar base de datos ni interfaz gráfica, ni construir mocks complicados de infraestructura	Añade una capa de indirección : interfaces y adaptadores adicionales que, en un proyecto pequeño, pueden sentirse como sobre-ingeniería
Cambiar de tecnología de infraestructura (motor de base de datos, framework web) no obliga a tocar el negocio	Requiere disciplina del equipo para no "filtrar" detalles de infraestructura de vuelta hacia el núcleo
El mismo núcleo puede exponerse a través de varios adaptadores distintos a la vez (API REST, cola de eventos, CLI)	Para dominios muy simples, el beneficio de aislar el núcleo puede no compensar el código adicional

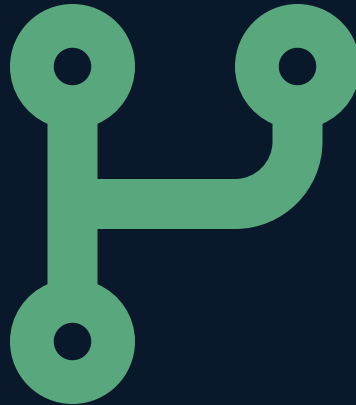
TENSIÓN CONCEPTUAL

Dos escalas distintas, no dos alternativas

SOA responde una pregunta a **nivel de sistema o de empresa**: cómo se organizan y se comunican entre sí muchos servicios independientes.

Hexagonal responde una pregunta a **nivel de código, dentro de un solo servicio o aplicación**: cómo proteger su lógica de negocio de los detalles técnicos que la rodean.

No compiten entre sí — son complementarios. Un servicio expuesto bajo SOA puede, perfectamente, estar construido por dentro con Arquitectura Hexagonal: su contrato hacia afuera es el "puerto" principal del hexágono, y la base de datos que usa por dentro es apenas uno de sus adaptadores.



INTERACCIÓN

¿SOA, Hexagonal, o ambos a la vez?

Para cada situación, decidan si aplica SOA, Arquitectura Hexagonal, o las dos combinadas, y justifiquen:

1. Una empresa necesita que su servicio de pagos lo puedan consumir tanto la app web como la app móvil como un socio externo, cada uno sin conocer los detalles internos del servicio de pagos.
2. Dentro de ese mismo servicio de pagos, el equipo quiere poder probar las reglas de negocio (validar un cobro, calcular comisiones) sin depender de la base de datos real.
3. El mismo equipo, un año después, decide migrar de una base de datos relacional a una NoSQL, y quiere que ese cambio no afecte la lógica de negocio del servicio.

5 minutos · respondan en voz alta o en el chat

SÍNTESIS

Ideas clave de hoy

1. **SOA** organiza el sistema como servicios de negocio reutilizables, con contrato explícito, bajo acoplamiento y autonomía — el "API Mandate" de Amazon (2002) forzó esta disciplina y, sin buscarlo, sentó las bases de AWS.
2. SOA da reutilización real entre equipos, pero exige gobernanza seria de contratos y versionado a medida que crecen los servicios.
3. **Arquitectura Hexagonal (Ports and Adapters)**, de Alistair Cockburn (2005), aísla el núcleo de negocio de la infraestructura mediante puertos (interfaces del dominio) y adaptadores (implementaciones concretas), con inversión de dependencias hacia el núcleo.
4. Hexagonal facilita probar el dominio de forma aislada y cambiar de infraestructura sin tocar el negocio, a cambio de una capa extra de indirección que puede no valer la pena en proyectos pequeños.
5. **SOA y Hexagonal operan en escalas distintas y son complementarios:** SOA organiza servicios a nivel de sistema; Hexagonal organiza el código dentro de cada servicio.

CIERRE

Para la próxima semana

- Repasar los diez patrones del catálogo POSA vistos hasta ahora en la Unidad 2 y ubicar, en cada uno, un ejemplo real o del proyecto de Fábrica Escuela.
- Identificar si algún servicio del proyecto de Fábrica Escuela podría beneficiarse de separar su núcleo de negocio de su infraestructura, al estilo de puertos y adaptadores.

La Semana 9 cierra la Unidad 2 con Microservicios y WebHooks: cómo SOA evolucionó hacia servicios más pequeños e independientes, y cómo se comunican mediante eventos.

